

Hidden Markov Models

Yongmei Tan

ymtan@bupt.edu.cn

HMMs在NLP中的应用 "词性标注"

Outline

- The “Fair Bet Casino”
- Hidden Markov Model
- Evaluation Problem
- Learning Problem
- Decoding Problem

the “Fair Bet Casino”

- *“The Fair Bet Casino”*
- The game is to flip coins, which results in only two possible outcomes: **Head** or **Tail**.
- The **Fair** coin will give **Heads** and **Tails** with same probability $\frac{1}{2}$.
- The **Biased** coin will give **Heads** with prob. $\frac{3}{4}$.

The “Fair Bet Casino” (cont’d)

- Thus, we define the probabilities:
 - $P(H|F) = P(T|F) = \frac{1}{2}$
 - $P(H|B) = \frac{3}{4}, P(T|B) = \frac{1}{4}$
 - The crooked dealer chages between Fair and Biased coins with probability 10%

The Fair Bet Casino Problem

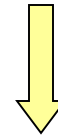
- **Input:** A sequence $x = x_1 x_2 x_3 \dots x_n$ of coin tosses made by two possible coins (**F** or **B**).
- **Output:** A sequence $\pi = \pi_1 \pi_2 \pi_3 \dots \pi_n$, with each π_i being either **F** or **B** indicating that x_i is the result of tossing the Fair or Biased coin respectively.

Problem...

Fair Bet Casino Problem

Any observed outcome of coin tosses could have been generated by any sequence of states!

Need to incorporate a way to grade different sequences differently.



Decoding Problem

$P(x|\text{fair coin})$ vs. $P(x|\text{biased coin})$

- Suppose first that dealer never changes coins. Some definitions:
 - $P(x|\text{fair coin})$: prob. of the dealer using the F coin and generating the outcome x .
 - $P(x|\text{biased coin})$: prob. of the dealer using the B coin and generating outcome x .

$P(x|\text{fair coin})$ vs. $P(x|\text{biased coin})$

- $P(x|\text{fair coin}) = P(x_1 \dots x_n | \text{fair coin})$

$$\prod_{i=1, n} p(x_i | \text{fair coin}) = (1/2)^n$$

- $P(x|\text{biased coin}) = P(x_1 \dots x_n | \text{biased coin}) =$

$$\prod_{i=1, n} p(x_i | \text{biased coin}) = (3/4)^k (1/4)^{n-k} = 3^k / 4^n$$

- k - the number of **H**eads in x .

$P(x|\text{fair coin})$ vs. $P(x|\text{biased coin})$

$$P(x|\text{fair coin}) = P(x|\text{biased coin})$$

$$1/2^n = 3^k/4^n$$

$$2^n = 3^k$$

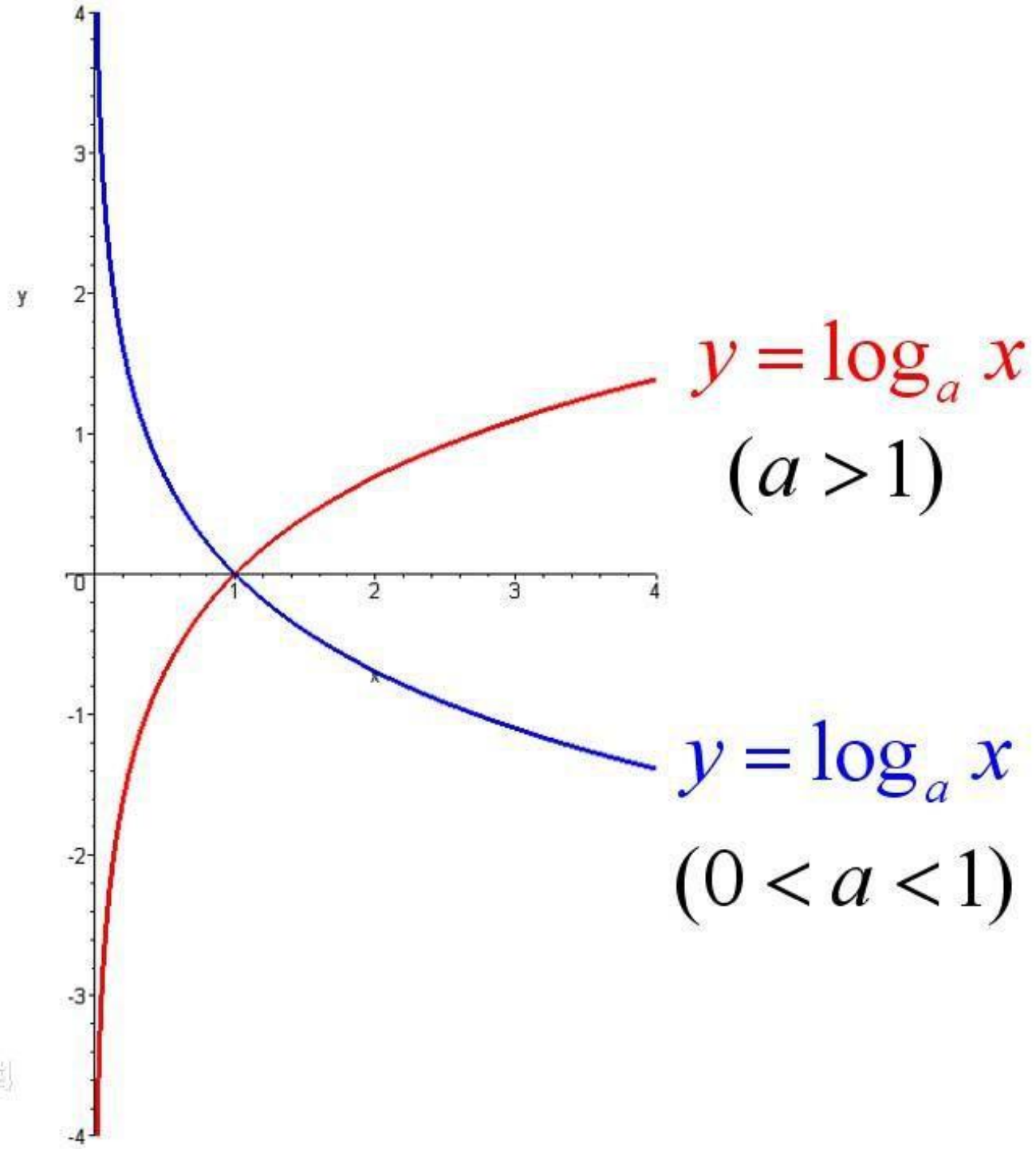
$$n = k \log_2 3$$

when $k = n / \log_2 3$ ($k \sim 0.67n$)

Log-odds Ratio

- We define *log-odds ratio* 对数几率比 as follows:

$$\begin{aligned}\log_2(P(x|\text{fair coin}) / P(x|\text{biased coin})) \\ &= \sum_{i=1}^k \log_2(p^+(x_i) / p^-(x_i)) \\ &= n - k \log_2 3\end{aligned}$$



Computing Log-odds Ratio in Sliding Windows

$$x_1 x_2 \boxed{x_3 x_4 x_5 x_6 x_7} x_8 \dots x_n$$

Consider a *sliding window* of the outcome sequence. Find the log-odds for this short window.

0



Disadvantages:

- different windows may classify the same position differently

Hidden Markov Model (HMM)

- Can be viewed as an abstract machine with k *hidden* states that emits symbols from an alphabet Σ .
- Each state has its own probability distribution, and the machine switches between states according to this probability distribution.
- While in a certain state, the machine makes 2 decisions:
 - What state should I move to next?
 - What symbol - from the alphabet Σ - should I emit?

Why “Hidden”?

- Observers can see the emitted symbols of an HMM but have *no ability to know which state the HMM is currently in*.
- Thus, the goal is to infer the most likely hidden states of an HMM based on the given sequence of emitted symbols.

HMM Parameters

Σ : set of emission characters.

Ex.: $\Sigma = \{H, T\}$ for coin tossing

$\Sigma = \{1, 2, 3, 4, 5, 6\}$ for dice 骰子
tossing

Q : set of hidden states, each emitting symbols
from Σ .

$Q = \{F, B\}$ for coin tossing

HMM Parameters (cont'd)

$A = (a_{kl})$: a $|Q| \times |Q|$ matrix of probability of changing from state k to state l .

$$a_{FF} = 0.9 \quad a_{FB} = 0.1$$

$$a_{BF} = 0.1 \quad a_{BB} = 0.9$$

$E = (e_k(b))$: a $|Q| \times |\Sigma|$ matrix of probability of emitting symbol b while being in state k .

$$e_F(0) = 1/2 \quad e_F(1) = 1/2$$

$$e_B(0) = 1/4 \quad e_B(1) = 3/4$$

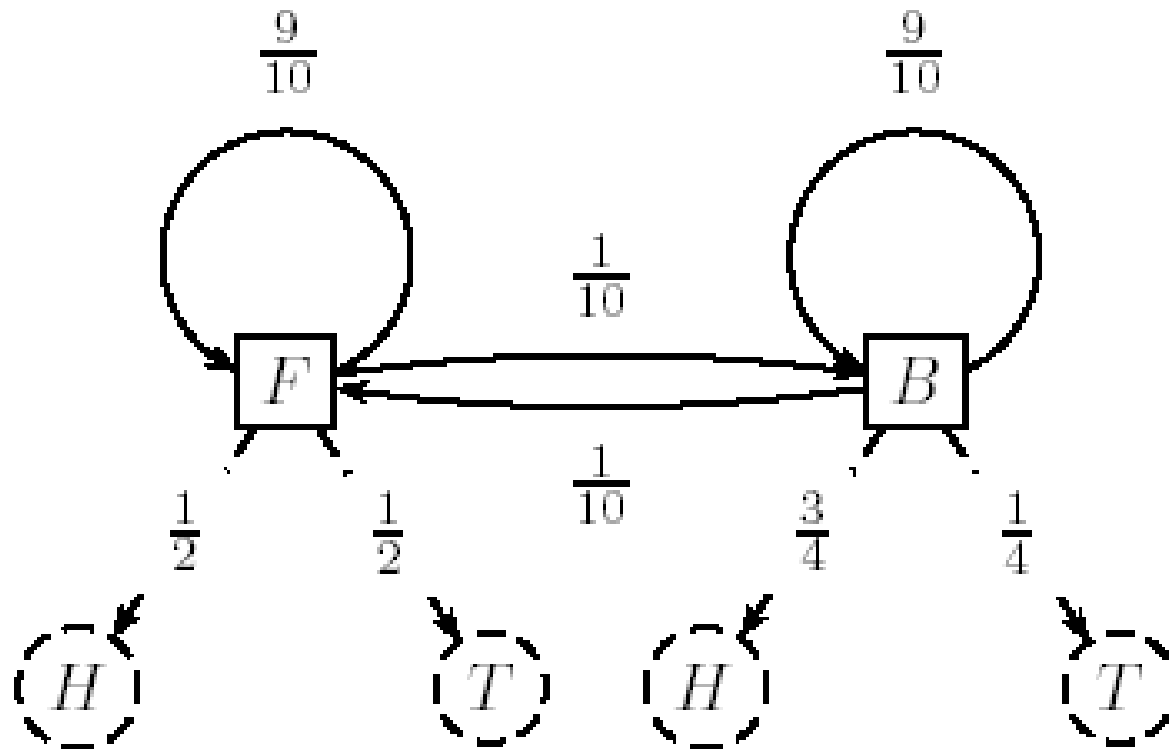
HMM for Fair Bet Casino

- The *Fair Bet Casino* in *HMM* terms:
 $\Sigma = \{0, 1\}$ (**0** for **T**ails and **1** **H**eads)
 $Q = \{F, B\}$ – *F* for Fair & *B* for Biased coin.
- Transition Probabilities *A* *** Emission Probabilities *E*

	Fair	Biased
Fair	$a_{FF} = 0.9$	$a_{FB} = 0.1$
Biased	$a_{BF} = 0.1$	$a_{BB} = 0.9$

	Tails(0)	Heads(1)
Fair	$e_F(0) = \frac{1}{2}$	$e_F(1) = \frac{1}{2}$
Biased	$e_B(0) = \frac{1}{4}$	$e_B(1) = \frac{3}{4}$

HMM for Fair Bet Casino (cont'd)



HMM model for the *Fair Bet Casino* Problem

Hidden Paths

- A *path* $\pi = \pi_1 \dots \pi_n$ in the HMM is defined as a sequence of states.
- Consider path $\pi = \text{FFFB BBBB FFF}$ and sequence $x = 01011101001$

Probability that x_i was emitted from state π_i

x	0	1	0	1	1	1	0	1	0	0	1
π	F	F	F	B	B	B	B	B	F	F	F
$P(x_i \pi_i)$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{1}{4}$	$\frac{3}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$
$P(\pi_{i-1} \rightarrow \pi_i)$	$\frac{1}{2}$	$\frac{9}{10}$	$\frac{9}{10}$	$\frac{1}{10}$	$\frac{9}{10}$	$\frac{9}{10}$	$\frac{9}{10}$	$\frac{9}{10}$	$\frac{1}{10}$	$\frac{9}{10}$	$\frac{9}{10}$

Transition probability from state π_{i-1} to state π_i

Main issues using HMMs

- 1. Evaluation problem. Given the HMM $M=(A, B, \pi)$ and the observation sequence $O=o_1 o_2 \dots o_K$, calculate the probability $P(O|M)$ that model M has generated sequence O .

$O=o_1 \dots o_K$ denotes a sequence of observations
 $o_K \in \{v_1, \dots, v_M\}$.

Main issues using HMMs

- 2. Learning problem. Given some training observation sequences $O=o_1 o_2 \dots o_K$ and general structure of HMM (numbers of hidden and visible states), determine HMM parameters $M=(A, B, \pi)$ that best fit training data.

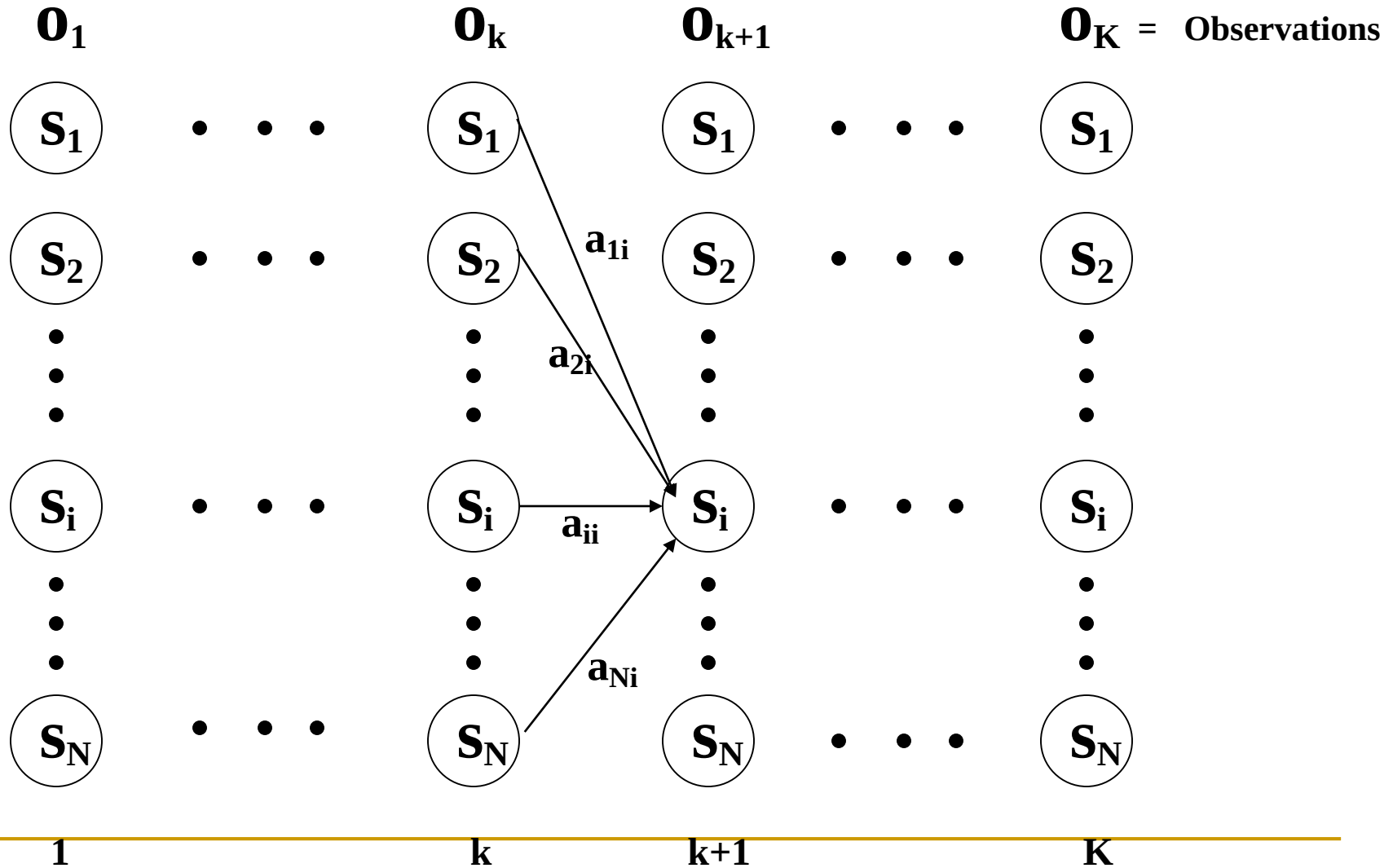
$O=o_1 \dots o_K$ denotes a sequence of observations $o_K \in \{v_1, \dots, v_M\}$.

Main issues using HMMs

- 3. Decoding problem. Given the HMM $M=(A, B, \pi)$ and the observation sequence $O=o_1 o_2 \dots o_K$, calculate the most likely sequence of hidden states s_i that produced this observation sequence O .

$O=o_1 \dots o_K$ denotes a sequence of observations $o_K \in \{v_1, \dots, v_M\}$.

Trellis representation of an HMM



1. Evaluation Problem

- Evaluation problem. Given the HMM $M=(A, B, \pi)$ and the observation sequence $O=o_1 o_2 \dots o_K$, calculate the probability that model M has generated sequence O .
- Trying to find probability of observations $O=o_1 o_2 \dots o_K$ by means of considering all hidden state sequences (as was done in example) is impractical:

N^K hidden state sequences - exponential complexity.

$$P(s_1 s_2 \dots s_K | M) = \pi_{i_1} a_{i_1 i_2} \dots a_{i_{K-1} i_K}$$

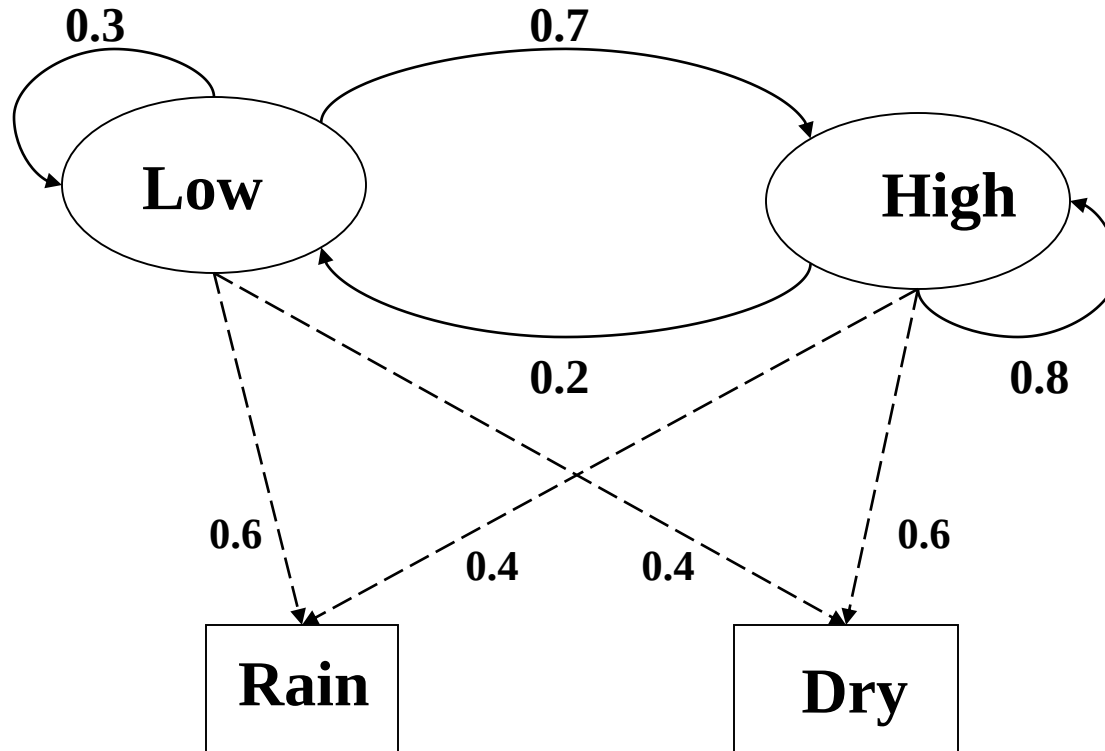
$$P(O|S, M) = b_{i_1}(o_1) b_{i_2}(o_2) \dots b_{i_K}(o_K)$$

$$P(O, S|M) = P(O|S, M) P(S|M) = \pi_{i_1} b_{i_1}(o_1) a_{i_1 i_2} b_{i_2}(o_2) \dots a_{i_{K-1} i_K} b_{i_K}(o_K)$$

$$P(O|M) = \sum_S P(O, S|M) = \sum_S P(O|S, M) P(S|M) = \sum_{i_1 i_2, \dots, i_K} \pi_{i_1} b_{i_1}(o_1) a_{i_1 i_2} b_{i_2}(o_2) \dots a_{i_{K-1} i_K} b_{i_K}(o_K)$$

$$O(KN^K)$$

Example: Calculation of observation sequence probability (cont'd)



Example: Calculation of observation sequence probability (cont'd)

- Two states : 'Low' and 'High' atmospheric pressure.
- Two observations : 'Rain' and 'Dry'.
- Transition probabilities:
 $P(\text{'Low'}|\text{'Low'})=0.3$, $P(\text{'High'}|\text{'Low'})=0.7$,
 $P(\text{'Low'}|\text{'High'})=0.2$, $P(\text{'High'}|\text{'High'})=0.8$
- Observation probabilities :
 $P(\text{'Rain'}|\text{'Low'})=0.6$, $P(\text{'Dry'}|\text{'Low'})=0.4$,
 $P(\text{'Rain'}|\text{'High'})=0.4$, $P(\text{'Dry'}|\text{'High'})=0.3$.
- Initial probabilities:
say $P(\text{'Low'})=0.4$, $P(\text{'High'})=0.6$.

Example: Calculation of observation sequence probability (cont'd)

- Suppose we want to calculate a probability of a sequence of observations in our example, {'Dry', 'Rain'}.
- Consider all possible hidden state sequences:
$$P(\text{'Dry', 'Rain'}) = P(\text{'Dry', 'Rain'}, \text{'Low', 'Low'}) + P(\text{'Dry', 'Rain'}, \text{'Low', 'High'}) + P(\text{'Dry', 'Rain'}, \text{'High', 'Low'}) + P(\text{'Dry', 'Rain'}, \text{'High', 'High'})$$

where first term is :

$$\begin{aligned} &P(\text{'Dry', 'Rain'}, \text{'Low', 'Low'}) = \\ &P(\text{'Dry', 'Rain'} \mid \text{'Low', 'Low'}) P(\text{'Low', 'Low'}) = \\ &P(\text{'Dry'} \mid \text{'Low'}) P(\text{'Rain'} \mid \text{'Low'}) P(\text{'Low'} \mid \text{'Low'}) P(\text{'Low'}) \\ &= 0.4 * 0.6 * 0.3 * 0.4 \end{aligned}$$

Example

- $M=(A, B, \pi)$ $S=\{1, 2, 3\}$ $O=\text{"red, white"}$

$$A = \begin{pmatrix} & 1 & 2 & 3 \\ 1 & 0.5 & 0.2 & 0.3 \\ 2 & 0.3 & 0.5 & 0.2 \\ 3 & 0.2 & 0.3 & 0.5 \end{pmatrix} \quad B = \begin{pmatrix} & \text{red} & \text{white} \\ 1 & 0.5 & 0.5 \\ 2 & 0.4 & 0.6 \\ 3 & 0.7 & 0.3 \end{pmatrix}$$

$$\pi = (0.2, 0.4, 0.4)^T$$

$$P(O|M)=?$$

- $P(\{\text{red, white}\}) =$
 $p(\{r, w\}, \{1,1\}) + p(\{r, w\}, \{1,2\}) + p(\{r, w\}, \{1,3\})$
 $+ p(\{r, w\}, \{2,1\}) + p(\{r, w\}, \{2,2\}) + p(\{r, w\},$
 $\{2,3\}) + p(\{r, w\}, \{3,1\}) + p(\{r, w\}, \{3,2\}) + p(\{r,$
 $w\}, \{3,3\}) = 0.248$

$$p(\{r, w\}, \{1,1\}) = p(\{r, w\} | \{1,1\}) * p(\{1,1\})$$

$$= 0.5 * 0.5 * 0.5 * 0.2 = 0.025$$

Forward-Backward HMM algorithms

- Use Forward-Backward HMM algorithms for efficient calculations.
- Define the forward variable $\alpha_k(i)$ as the joint probability of the partial observation sequence $o_1 o_2 \dots o_k$ and that the hidden state at time k is s_i :

$$\alpha_k(i) = P(o_1 o_2 \dots o_k, q_k = s_i)$$

Forward recursion for HMM

- Initialization:

$$\alpha_1(i) = P(o_1, q_1 = s_i) = \pi_i b_i(o_1), 1 \leq i \leq N.$$

- Forward recursion:

$$\begin{aligned} \alpha_{k+1}(i) &= P(o_1 o_2 \dots o_{k+1}, q_{k+1} = s_i) = \\ &\sum_j P(o_1 o_2 \dots o_{k+1}, q_k = s_j, q_{k+1} = s_i) = \\ &\sum_j P(o_1 o_2 \dots o_k, q_k = s_j) a_{ji} b_i(o_{k+1}) = \\ &[\sum_j \alpha_k(j) a_{ji}] b_i(o_{k+1}), 1 \leq j \leq N, 1 \leq k \leq K-1. \end{aligned}$$

- Termination:

$$P(o_1 o_2 \dots o_K | M) = \sum_i P(o_1 o_2 \dots o_K, q_K = s_i) = \sum_i \alpha_K(i), 1 \leq i \leq N.$$

- Complexity :

N^2K operations.

Example

- Use Forward recursion for HMM
- $M=(A, B, \pi)$ $S=\{1, 2, 3\}$ $K=3$ $O=\text{"red, white, red"}$

$$A = \begin{pmatrix} & 1 & 2 & 3 \\ 1 & 0.5 & 0.2 & 0.3 \\ 2 & 0.3 & 0.5 & 0.2 \\ 3 & 0.2 & 0.3 & 0.5 \end{pmatrix} \quad B = \begin{pmatrix} & \text{red} & \text{white} \\ 1 & 0.5 & 0.5 \\ 2 & 0.4 & 0.6 \\ 3 & 0.7 & 0.3 \end{pmatrix}$$

$$\pi = (0.2, 0.4, 0.4)^T$$

$$P(O|M)=?$$

解 按照算法 10.2

(1) 计算初值

$$\alpha_1(1) = \pi_1 b_1(o_1) = 0.10$$

$$\alpha_1(2) = \pi_2 b_2(o_1) = 0.16$$

$$\alpha_1(3) = \pi_3 b_3(o_1) = 0.28$$

(2) 递推计算

$$\alpha_2(1) = \left[\sum_{i=1}^3 \alpha_1(i) a_{i1} \right] b_1(o_2) = 0.154 \times 0.5 = 0.077$$

$$\alpha_2(2) = \left[\sum_{i=1}^3 \alpha_1(i) a_{i2} \right] b_2(o_2) = 0.184 \times 0.6 = 0.1104$$

$$\alpha_2(3) = \left[\sum_{i=1}^3 \alpha_1(i) a_{i3} \right] b_3(o_2) = 0.202 \times 0.3 = 0.0606$$

$$\alpha_3(1) = \left[\sum_{i=1}^3 \alpha_2(i) a_{i1} \right] b_1(o_3) = 0.04187$$

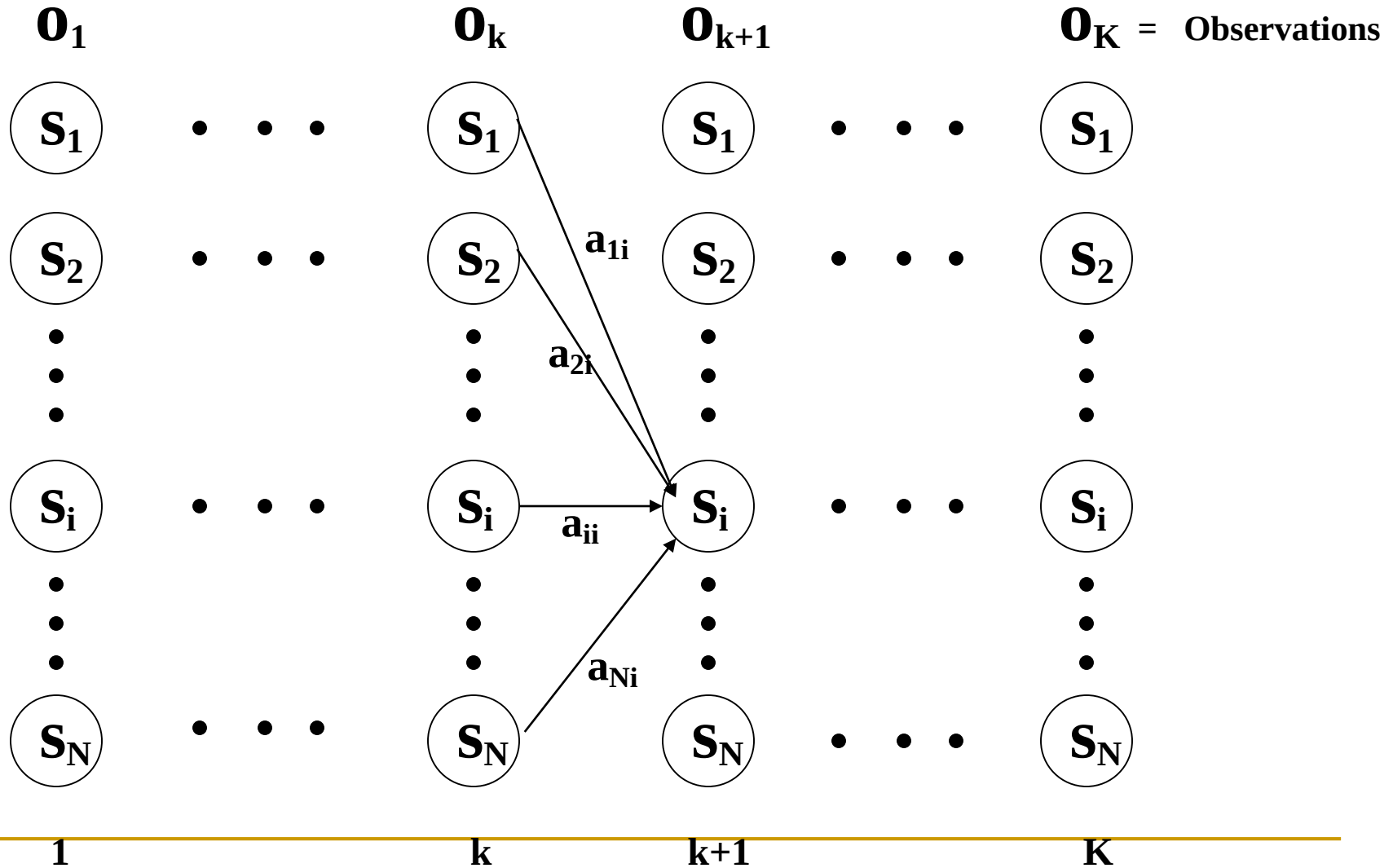
$$\alpha_3(2) = \left[\sum_{i=1}^3 \alpha_2(i) a_{i2} \right] b_2(o_3) = 0.03551$$

$$\alpha_3(3) = \left[\sum_{i=1}^3 \alpha_2(i) a_{i3} \right] b_3(o_3) = 0.05284$$

(3) 终止

$$P(O | \lambda) = \sum_{i=1}^3 \alpha_3(i) = 0.13022$$

Trellis representation of an HMM



Backward recursion for HMM

- Define the backward variable $\beta_k(i)$ as the joint probability of the partial observation sequence $o_{k+1} o_{k+2} \dots o_K$ given that the hidden state at time k is s_i :

$$\beta_k(i) = P(o_{k+1} o_{k+2} \dots o_K | q_k = s_i)$$

- Initialization:

$$\beta_K(i) = 1 \quad i=1, 2, \dots, N$$

- Backward recursion:

$$\begin{aligned} \beta_k(i) &= P(o_{k+1} o_{k+2} \dots o_K | q_k = s_i) = \\ &\sum_j P(o_{k+1} o_{k+2} \dots o_K, q_{k+1} = s_j | q_k = s_i) = \\ &\sum_j P(o_{k+2} o_{k+3} \dots o_K | q_{k+1} = s_j) a_{ij} b_j(o_{k+1}) = \\ &\sum_j \beta_{k+1}(j) a_{ij} b_j(o_{k+1}), \quad k=K-1, K-2, \dots, 1 \quad i=1, 2, \dots, N \end{aligned}$$

- Termination:

$$\begin{aligned} P(o_1 o_2 \dots o_K | M) &= \sum_i P(o_1 o_2 \dots o_K, q_1 = s_i) = \\ &\sum_i P(o_1 o_2 \dots o_K | q_1 = s_i) P(q_1 = s_i) = \sum_i \pi_i b_i(o_1) \beta_1(i) \end{aligned}$$

$$i=1, 2, \dots, N$$

Example

- Use Backward recursion for HMM
- $M=(A, B, \pi)$ $S=\{1, 2, 3\}$ $K=4$ $O=\text{"red, white, red, white"}$

$$A = \begin{pmatrix} & 1 & 2 & 3 \\ 1 & 0.5 & 0.2 & 0.3 \\ 2 & 0.3 & 0.5 & 0.2 \\ 3 & 0.2 & 0.3 & 0.5 \end{pmatrix} \quad B = \begin{pmatrix} & \text{red} & \text{white} \\ 1 & 0.5 & 0.5 \\ 2 & 0.4 & 0.6 \\ 3 & 0.7 & 0.3 \end{pmatrix}$$

$$\pi = (0.2, 0.4, 0.4)^T$$

$$P(O|M)=?$$

- Step 1:

- $\beta_4(1) = 1 \quad \beta_4(2) = 1 \quad \beta_4(3) = 1$
- $\beta_3(1) = \beta_4(1)*a_{11}*b_1(\text{white}) + \beta_4(2)*a_{12}*b_2(\text{white}) + \beta_4(3)*a_{13}*b_3(\text{white}) =$
 $1*0.5*0.5+1*0.2*0.6+1*0.3*0.3 = 0.46$
- $\beta_3(2) = \beta_4(1)*a_{21}*b_1(\text{white}) + \beta_4(2)*a_{22}*b_2(\text{white}) + \beta_4(3)*a_{23}*b_3(\text{white}) =$
 $1*0.3*0.5+1*0.5*0.6+1*0.2*0.3=0.51$
- $\beta_3(3) = \beta_4(1)*a_{31}*b_1(\text{white}) + \beta_4(2)*a_{32}*b_2(\text{white}) + \beta_4(3)*a_{33}*b_3(\text{white}) = 0.43$

- Step 2:

- $\beta_2(1) = \beta_3(1)*a_{11}*b_1(\text{red}) + \beta_3(2)*a_{12}*b_2(\text{red}) + \beta_3(3)*a_{13}*b_3(\text{red}) = 0.2461$
- $\beta_2(2) = \beta_3(1)*a_{21}*b_1(\text{red}) + \beta_3(2)*a_{22}*b_2(\text{red}) + \beta_3(3)*a_{23}*b_3(\text{red}) = 0.2312$
- $\beta_2(3) = \beta_3(1)*a_{31}*b_1(\text{red}) + \beta_3(2)*a_{32}*b_2(\text{red}) + \beta_3(3)*a_{33}*b_3(\text{red}) = 0.2577$

- Step 3:

- $\beta_1(1) = \beta_2(1)*a_{11}*b_1(\text{white}) + \beta_2(2)*a_{12}*b_2(\text{white}) + \beta_2(3)*a_{13}*b_3(\text{white}) = 0.112462$
- $\beta_1(2) = \beta_2(1)*a_{21}*b_1(\text{white}) + \beta_2(2)*a_{22}*b_2(\text{white}) + \beta_2(3)*a_{23}*b_3(\text{white}) = 0.121737$
- $\beta_1(3) = \beta_2(1)*a_{31}*b_1(\text{white}) + \beta_2(2)*a_{32}*b_2(\text{white}) + \beta_2(3)*a_{33}*b_3(\text{white}) = 0.104881$

- step 5:

- $$P(O|M) = \pi_1 * \beta_1(1) * b_1(\text{red}) + \pi_2 * \beta_1(2) * b_2(\text{red}) + \pi_3 * \beta_1(3) * b_3(\text{red}) = 0.060091$$

2. Learning problem

- Learning problem. Given some training observation sequences $O = o_1 o_2 \dots o_K$ and general structure of HMM (numbers of hidden and visible states), determine HMM parameters $M = (A, B, \pi)$ that best fit training data, that is maximizes $P(O | M)$.

2. Learning problem

- If training data has information about sequence of hidden states (as in word recognition example), then use maximum likelihood estimation of parameters (supervised method):

$$a_{ij} = P(s_j | s_i) = \frac{\text{Number of transitions from state } s_i \text{ to state } s_j}{\text{Number of transitions out of state } s_i}$$

$$b_i(o_m) = P(o_m | s_i) = \frac{\text{Number of times observation } o_m \text{ occurs in state } s_i}{\text{Number of times in state } s_i}$$

2. Learning problem

- There is no algorithm producing optimal parameter values.
- Use iterative expectation-maximization algorithm to find local maximum of $P(O | M)$ - Baum-Welch algorithm. (unsupervised learning method)

Baum-Welch algorithm

General idea:

$$\mathbf{a}_{ij} = \mathbf{P}(s_j | s_i) = \frac{\text{Expected number of transitions from state } s_i \text{ to state } s_j}{\text{Expected number of transitions out of state } s_i}$$

$$\mathbf{b}_i(\mathbf{o}_m) = \mathbf{P}(\mathbf{o}_m | s_i) = \frac{\text{Expected number of times observation } \mathbf{o}_m \text{ occurs in state } s_i}{\text{Expected number of times in state } s_i}$$

$$\pi_i = \mathbf{P}(s_i) = \text{Expected frequency in state } S_i \text{ at time } k=1.$$

Baum-Welch algorithm: expectation step

- Define variable $\xi_k(i,j)$ as the probability of being in state s_i at time k and in state s_j at time $k+1$, given the observation sequence $o_1 o_2 \dots o_K$.

$$\xi_k(i,j) = P(\mathbf{q}_k = s_i, \mathbf{q}_{k+1} = s_j \mid \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_K)$$

$$\xi_k(i,j) = \frac{P(\mathbf{q}_k = s_i, \mathbf{q}_{k+1} = s_j, \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_K)}{P(\mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_K)} =$$

$$\frac{P(\mathbf{q}_k = s_i, \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_k) a_{ij} b_j(\mathbf{o}_{k+1}) P(\mathbf{o}_{k+2} \dots \mathbf{o}_K \mid \mathbf{q}_{k+1} = s_j)}{\sum_i \sum_j P(\mathbf{q}_k = s_i, \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_k) a_{ij} b_j(\mathbf{o}_{k+1}) P(\mathbf{o}_{k+2} \dots \mathbf{o}_K \mid \mathbf{q}_{k+1} = s_j)} =$$
$$\frac{\alpha_k(i) a_{ij} b_j(\mathbf{o}_{k+1}) \beta_{k+1}(j)}{\sum_i \sum_j \alpha_k(i) a_{ij} b_j(\mathbf{o}_{k+1}) \beta_{k+1}(j)}$$

Baum-Welch algorithm: expectation step

- Define variable $\gamma_k(i)$ as the probability of being in state s_i at time k , given the observation sequence $o_1 o_2 \dots o_K$.

$$\gamma_k(i) = \mathbf{P}(\mathbf{q}_k = s_i \mid \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_K)$$

$$\gamma_k(i) = \frac{\mathbf{P}(\mathbf{q}_k = s_i, \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_K)}{\mathbf{P}(\mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_K)} = \frac{\alpha_k(i) \beta_k(i)}{\sum_i \alpha_k(i) \beta_k(i)}$$

Baum-Welch algorithm: expectation step

- We calculated $\xi_k(\mathbf{i}, \mathbf{j}) = P(\mathbf{q}_k = s_i, \mathbf{q}_{k+1} = s_j \mid \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_K)$
and $\gamma_k(\mathbf{i}) = P(\mathbf{q}_k = s_i \mid \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_K)$
- Expected number of transitions from state s_i to state s_j
 $= \sum_k \xi_k(\mathbf{i}, \mathbf{j})$
- Expected number of transitions out of state $s_i = \sum_k \gamma_k(\mathbf{i})$
- Expected number of times observation v_m occurs in state $s_i = \sum_k \gamma_k(\mathbf{i})$, k is such that $o_k = v_m$
- Expected frequency in state s_i at time $k=1$: $\gamma_1(\mathbf{i})$.

Baum-Welch algorithm: maximization step

$$\mathbf{a}_{ij} = \frac{\text{Expected number of transitions from state } S_i \text{ to state } S_j}{\text{Expected number of transitions out of state } S_i} = \frac{\sum_k \xi_{k(i,j)}}{\sum_k \gamma_{k(i)}}$$

$$\mathbf{b}_i(\mathbf{o}_m) = \frac{\text{Expected number of times observation } \mathbf{o}_m \text{ occurs in state } S_i}{\text{Expected number of times in state } S_i} = \frac{\sum_{k, \mathbf{o}_k = \mathbf{o}_m} \gamma_{k(i)}}{\sum_k \gamma_{k(i)}}$$

$$\pi_i = (\text{Expected frequency in state } S_i \text{ at time } k=1) = \gamma_1(i).$$

3. Decoding problem

- Decoding problem. Given the HMM $M=(A, B, \pi)$ and the observation sequence $O=o_1 o_2 \dots o_K$, calculate the most likely sequence of hidden states s_i that produced this observation sequence.
- We want to find the state sequence $Q=q_1 \dots q_K$ which maximizes $P(Q \mid o_1 o_2 \dots o_K)$, or equivalently $P(Q, o_1 o_2 \dots o_K)$.
- Brute force consideration of all paths takes exponential time. Use efficient Viterbi algorithm instead.
- Define variable $\delta_k(i)$ as the maximum probability of producing observation sequence $o_1 o_2 \dots o_k$ when moving along any hidden state sequence $q_1 \dots q_{k-1}$ and getting into $q_k = s_i$.
$$\delta_k(i) = \max P(q_1 \dots q_{k-1}, q_k = s_i, o_1 o_2 \dots o_k)$$
where max is taken over all possible paths $q_1 \dots q_{k-1}$.

Viterbi algorithm

- General idea:

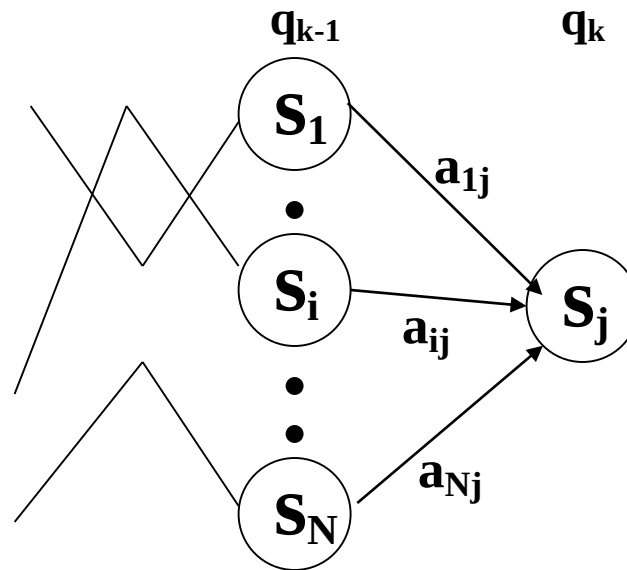
根据动态规划原理，最优路径具有这样的特性：如果最优路径在时刻 k 通过结点 S_k^* ，那么这一路径从结点 S_k^* 到终点 S_K^* 的部分路径，对于从 S_k^* 到 S_K^* 的所有可能的部分路径来说，必须是最优的。

如果不是这样，那么从 S_k^* 到 S_K^* 就有另一条更好的部分路径存在，如果把它和从 S_k^* 到 S_K^* 的部分路径连接起来，就会形成一条比原来的路径更优的路径，这是矛盾的。

Viterbi algorithm

- 从时刻 $k=1$ 开始，递推地计算在时刻 k 转态为 s_K 的各条部分路径的最大概率，直到得到时刻 $k=K$ 状态为 s_K 的各条路径的最大概率。
- 时刻 $k=K$ 的最大概率即为最优路径的概率 P^* ，最优路径的终结点 s_K^* 也同时得到。
- 找出最优路径的各个结点，从终结点 s_K^* 开始，由后向前逐步求得结点 $s_{K-1}^*, \dots, s_1^*$, 得到最优路径
 $s_1^* s_2^* \dots s_K^*$

Viterbi algorithm



- $\delta_k(j) = \max P(q_1 \dots q_{k-1}, q_k = s_j, o_1 o_2 \dots o_k)$
 $= \max_i [\max P(q_1 \dots q_{k-1} = s_i, o_1 o_2 \dots o_{k-1}) a_{ij} b_j(o_k)]$
 $= \max_i [\delta_{k-1}(i) a_{ij}] b_j(o_k)$

- $\psi_k(j) = \arg \max_{1 \leq i \leq N} \delta_{k-1}(i) a_{ij}$

Viterbi algorithm

- Initialization:

$$\delta_1(i) = P(q_1 = s_i, o_1) = \pi_i b_i(o_1), 1 \leq i \leq N.$$

$$\psi_1(i) = 0, 1 \leq i \leq N.$$

- Forward recursion: $2 \leq k \leq K$

$$\begin{aligned} \delta_k(i) &= \max P(q_1 \dots q_{k-1}, q_k = s_i, o_1 o_2 \dots o_k) = \\ &= \max_j [\max P(q_1 \dots q_{k-1} = s_j, o_1 o_2 \dots o_{k-1}) a_{ji} b_i(o_k)] = \\ &= \max_j [\delta_{k-1}(j) a_{ji}] b_i(o_k) \quad 1 \leq i \leq N \end{aligned}$$

$$\psi_k(i) = \arg \max_j [\delta_{k-1}(j) a_{ji}] \quad 1 \leq i \leq N$$

- Termination: choose best path ending at time K

$$P^* = \max_i [\delta_K(i)] \quad i_K^* = \arg \max_i [\delta_K(i)] \quad 1 \leq i \leq N$$

- Backtrack best path.

$$i_k^* = \psi_{k+1}(i_{k+1}^*) \quad k = K-1, K-2, \dots, 1$$

$$\text{best path } I^* = (i_1^*, i_2^*, \dots, i_K^*)$$

This algorithm is similar to the forward recursion of evaluation problem, with

Σ replaced by max and additional backtracking.

Example

- $M=(A, B, \pi)$ $S=\{1, 2, 3\}$ $O=\{\text{red, white, red}\},$

$$A = \begin{pmatrix} & 1 & 2 & 3 \\ 1 & 0.5 & 0.2 & 0.3 \\ 2 & 0.3 & 0.5 & 0.2 \\ 3 & 0.2 & 0.3 & 0.5 \end{pmatrix} \quad B = \begin{pmatrix} & \text{red} & \text{white} \\ 1 & 0.5 & 0.5 \\ 2 & 0.4 & 0.6 \\ 3 & 0.7 & 0.3 \end{pmatrix}$$

$$\pi = (0.2, 0.4, 0.4)^T$$

What is the path that maximizes $P(S|O)$ over all possible paths?

解 如图 10.4 所示, 要在所有可能的路径中选择一条最优路径, 按照以下步骤处理:

(1) 初始化. 在 $t=1$ 时, 对每一个状态 i , $i=1,2,3$, 求状态为 i 观测 o_1 为红的概率, 记此概率为 $\delta_1(i)$, 则

$$\delta_1(i) = \pi_i b_i(o_1) = \pi_i b_i(\text{红}), \quad i=1,2,3$$

代入实际数据

$$\delta_1(1) = 0.10, \quad \delta_1(2) = 0.16, \quad \delta_1(3) = 0.28$$

记 $\psi_1(i) = 0, \quad i=1,2,3$.

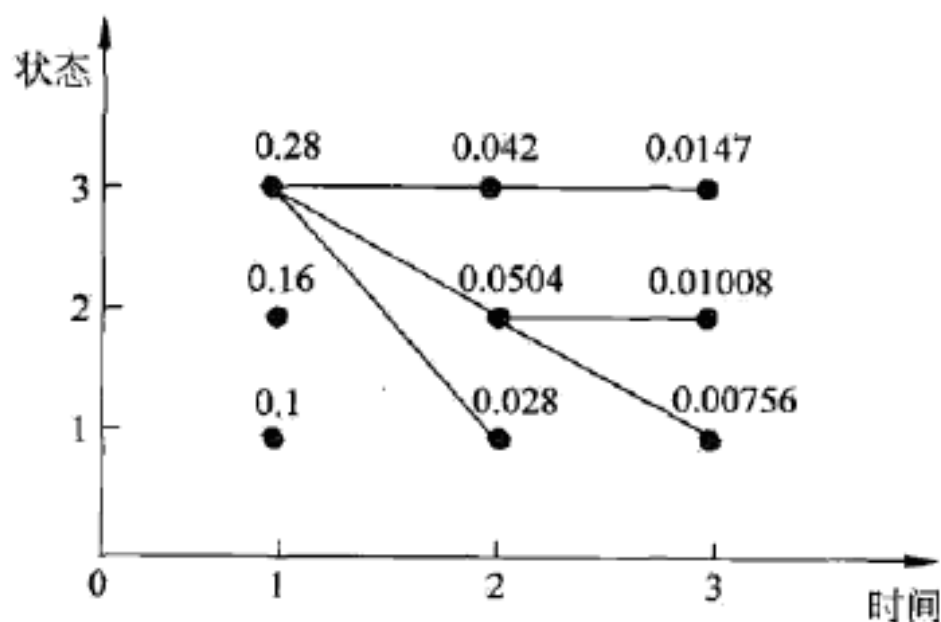


图 10.4 求最优路径

(2) 在 $t=2$ 时, 对每个状态 i , $i=1,2,3$, 求在 $t=1$ 时状态为 j 观测为红并在 $t=2$ 时状态为 i 观测 o_2 为白的路径的最大概率, 记此最大概率为 $\delta_2(i)$, 则

$$\delta_2(i) = \max_{1 \leq j \leq 3} [\delta_1(j)a_{ji}]b_i(o_2)$$

同时, 对每个状态 i , $i=1,2,3$, 记录概率最大路径的前一个状态 j :

$$\psi_2(i) = \arg \max_{1 \leq j \leq 3} [\delta_1(j)a_{ji}], \quad i=1,2,3$$

计算:

$$\begin{aligned} \delta_2(1) &= \max_{1 \leq j \leq 3} [\delta_1(j)a_{j1}]b_1(o_2) \\ &= \max_j \{0.10 \times 0.5, 0.16 \times 0.3, 0.28 \times 0.2\} \times 0.5 \\ &= 0.028 \end{aligned}$$

$$\psi_2(1) = 3$$

$$\delta_2(2) = 0.0504, \quad \psi_2(2) = 3$$

$$\delta_2(3) = 0.042, \quad \psi_2(3) = 3$$

同样, 在 $t=3$ 时,

$$\delta_3(i) = \max_{1 \leq j \leq 3} [\delta_2(j)a_{ji}]b_i(o_3)$$

$$\psi_3(i) = \arg \max_{1 \leq j \leq 3} [\delta_2(j)a_{ji}]$$

$$\delta_3(1) = 0.00756, \quad \psi_3(1) = 2$$

$$\delta_3(2) = 0.01008, \quad \psi_3(2) = 2$$

$$\delta_3(3) = 0.0147, \quad \psi_3(3) = 3$$

(3) 以 P^* 表示最优路径的概率, 则

$$P^* = \max_{1 \leq i \leq 3} \delta_3(i) = 0.0147$$

最优路径的终点是 i_3^* :

$$i_3^* = \arg \max_i [\delta_3(i)] = 3$$

(4) 由最优路径的终点 i_3^* , 逆向找到 i_2^*, i_1^* :

$$\text{在 } t=2 \text{ 时, } i_2^* = \psi_3(i_3^*) = \psi_3(3) = 3$$

$$\text{在 } t=1 \text{ 时, } i_1^* = \psi_2(i_2^*) = \psi_2(3) = 3$$

于是求得最优路径, 即最优状态序列 $I^* = (i_1^*, i_2^*, i_3^*) = (3, 3, 3)$.

Homework 1

- 10.1 $M=(A, B, \pi)$ $S=\{1, 2, 3\}$ $O=\{\text{white, red, white, red}\}$

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 0.5 & 0.2 & 0.3 \\ 2 & 0.3 & 0.5 & 0.2 \\ 3 & 0.2 & 0.3 & 0.5 \end{pmatrix} \quad B = \begin{pmatrix} \text{red} & \text{white} \\ 1 & 0.5 & 0.5 \\ 2 & 0.4 & 0.6 \\ 3 & 0.7 & 0.3 \end{pmatrix}$$

$$\pi = (0.2, 0.4, 0.4)^\top$$

$P(O|M)=?$ (Backward recursion)

Homework 2

- 10.2 $M=(A, B, \pi)$ $S=\{1, 2, 3\}$ $O=\{\text{red, white, red, red, white, red, white, white}\}$

$$A = \begin{pmatrix} & 1 & 2 & 3 \\ 1 & 0.5 & 0.1 & 0.4 \\ 2 & 0.3 & 0.5 & 0.2 \\ 3 & 0.2 & 0.2 & 0.6 \end{pmatrix} \quad B = \begin{pmatrix} & \text{red} & \text{white} \\ 1 & 0.5 & 0.5 \\ 2 & 0.4 & 0.6 \\ 3 & 0.7 & 0.3 \end{pmatrix}$$

$$K=8, \pi=(0.2, 0.3, 0.5)^T$$

$P(q_4=s_3|O,M)=?$ (Forward Backward recursion)

Homework 3

- 10.3 $M=(A, B, \pi)$ $S=\{1, 2, 3\}$ $O=\{\text{red, white, red, white}\}$

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 0.5 & 0.2 & 0.3 \\ 2 & 0.3 & 0.5 & 0.2 \\ 3 & 0.2 & 0.3 & 0.5 \end{pmatrix} \quad B = \begin{pmatrix} \text{red} & \text{white} \\ 1 & 0.5 & 0.5 \\ 2 & 0.4 & 0.6 \\ 3 & 0.7 & 0.3 \end{pmatrix}$$

$$\pi = (0.2, 0.4, 0.4)^\top$$

$$Q^* = (q_1^*, q_2^*, q_3^*, q_4^*) \text{ (Viterbi algorithm)}$$

References

- Rabiner L, Juang B. An introduction to hidden markov models. IEEE ASSP Magazine, January 1986.
- Rabiner L. A tutorial on hidden markov models and selected applications in speech recognition. Proceedings of IEEE, 1989.



Thank You!

